

04e — GraphQL

Key Concepts

Term	Definition
GraphQL	Query language for APIs where the client specifies exactly what data it needs
Schema	Strongly typed definition of all data types and operations available
Resolver	Server-side function that fetches data for a specific field
Query	Read operation (like GET)
Mutation	Write operation — create, update, delete
Subscription	Real-time streaming (uses WebSockets)
Over-fetching	Getting more data than needed (REST problem)
Under-fetching	Needing multiple requests to get all data (REST problem)
DataLoader	Batching library that collects field lookups and fires one DB query instead of N
Persisted Queries	Sending a hash of the query instead of the full text — enables GET requests → CDN caching

How It Works

1. Client sends a **POST** `/graphql` request with a query in the body
2. Query describes the exact shape of data needed (fields, nested types)
3. Server parses the query against the Schema
4. For each field, the matching Resolver runs and fetches data
5. Server assembles and returns exactly the requested shape — nothing more

```
query {  
  user(id: "123") {  
    name  
    posts {  
      title  
    }  
  }  
}
```

Three Operations

Operation	Purpose	REST Equivalent
-----------	---------	-----------------

Operation	Purpose	REST Equivalent
query	Read data	GET
mutation	Create / update / delete	POST / PUT / DELETE
subscription	Real-time push	WebSockets

N+1 Problem in GraphQL

- **Problem:** Fetching 100 users with a `country` field → 1 query for users + 100 queries for country = 101 total
- **Root cause:** Each resolver runs independently, doesn't know about sibling resolvers
- **Fix:** DataLoader — batches all country IDs collected in one tick → fires `SELECT * FROM countries WHERE id IN (...)` → 2 queries total
- **Where the fix lives:** Server side, not the client query

Caching in GraphQL

Strategy	How
Client-side (Apollo/urql)	Cache responses in memory keyed by query + variables
Persisted Queries	Hash the query → use GET request → CDN can cache
Field-level hints	<code>@cacheControl(maxAge: 3600)</code> per field on the server

- HTTP caching doesn't work out of the box — all requests are POST to same URL
- CDN cannot distinguish between different GraphQL queries

GraphQL vs REST

	REST	GraphQL
Endpoints	Many	One (<code>/graphql</code>)
Data shape control	Server	Client
Over-fetching	Common	Eliminated
Caching	Easy (HTTP/CDN)	Manual effort required
Versioning	<code>/v1/</code> , <code>/v2/</code>	Additive schema changes
Best for	Public APIs, simple CRUD, cache-heavy	Complex data, mobile clients, flexible frontends

Industry Examples

- **GitHub API v4** — GraphQL replaces 5+ REST calls for dashboard data with one query
 - **Facebook** — built GraphQL for mobile; slow networks need minimal payloads
 - **Twitter/X** — uses GraphQL internally for mobile app efficiency
 - **Shopify** — uses persisted queries for CDN caching in GraphQL
-

When NOT to Use GraphQL

- **Cache-heavy public content** — news sites, blogs: REST GET URLs cache at CDN; GraphQL POSTs don't
 - **Simple CRUD apps** — GraphQL's complexity isn't justified
 - **Public third-party APIs** — developers expect REST conventions
-

Interview Questions

Q: What is GraphQL and how is it different from REST? A: Query language where the client specifies exactly what fields it needs; uses one endpoint vs many; eliminates over/under-fetching.

Q: What is the N+1 problem in GraphQL and how do you fix it? A: Each field resolver runs independently, causing N extra DB queries per parent record. Fix: DataLoader batches all lookups into one query server-side.

Q: How do you handle caching in GraphQL? A: HTTP caching doesn't work (all POSTs to same URL).

Options: Apollo client-side cache, persisted queries (convert to GET for CDN caching), field-level

[@cacheControl](#) hints.

Q: When would you NOT use GraphQL? A: When you need heavy CDN caching (news sites, public content) — REST GET endpoints cache naturally; GraphQL POST requests don't.

Q: What is a resolver? A: A server-side function that fetches data for a specific field in the schema. When a client queries `user.name`, the `name` resolver runs to return that value.